# Cybersecurity Hackathon

**PROBLEM STATEMENT, RULES AND REGULATIONS**

24 Hour Cybersecurity Hackathon | IIITDM Kancheepuram

Cybersecurity Club

## Rules and Regulations

| | | | |
|---|---|---|---|
| **Development starts** | Saturday, 31/10/2026 | 9:00 PM | Remote |
| **Final submission deadline** | Sunday, 01/11/2026 | 9:00 PM | Online (Google Form) |

### 1. Time Limit

The hackathon runs for 24 hours, from Saturday, 31/10/2026 to Sunday, 01/11/2026. Teams should plan their work to fit within this period.

### 2. Remote Work

All teams work remotely. No on-site presence is required during development.

### 3. Teams

Teams can have 1 to 3 members. Each participant can be part of only one team.

### 4. Cut-off

Every project must be submitted strictly before 9:00 PM on 01/11/2026. Submissions made after the deadline will not be accepted, and any commits pushed to the repository after the deadline will be ignored.

### 5. How to Submit

Only the team leader submits the project by 1st November, using the Google Form: Cybersecurity Hackathon submission

## Problem Statement

**NETWORK TRACK**

# Diagnosing Network Problems from Packet Captures

**CONTEXT**

Small offices, labs and campus departments depend on their network every day, but most of them have no dedicated network engineer or security person. When the network is slow, drops connections or behaves strangely, the usual way to find the cause is to open a packet capture and read it line by line. This takes time, needs deep protocol knowledge and is very hard for students, beginners and small teams. It is even harder to tell a harmless fault from an attack hiding in the same traffic.

### PROBLEM

Build a tool that takes a packet capture file as input, finds the faults and attacks hidden in the traffic and explains them in plain language, along with the likely cause and a suggested fix. Teams are free to choose their own approach, language and interface (command line, web dashboard or desktop app).

### CHALLENGE POINTS

- **Reading captures safely:** Load capture files and handle large, broken or malicious input without crashing or leaking data.
- **Finding faults:** Detect common problems such as TCP retransmissions and resets, DNS failures, duplicate IP addresses, ARP conflicts, MTU mismatches and unusual delays.
- **Finding attacks:** Detect suspicious activity such as port scans, ARP spoofing, DNS hijacking and rogue DHCP servers.
- **Telling an outage from an attack:** Both look like “the network is broken” to a user. Separate a plain fault from malicious activity, and keep false alarms low.
- **Explaining clearly:** Turn technical findings into simple language that a beginner can follow.
- **Recommending fixes:** Suggest the likely cause and a practical next step for every problem found.
- **Showing the evidence:** Point to the exact packets or conversations behind each diagnosis, for example with a timeline or summary view.

### IMPACT

Without a tool like this, small networks suffer in three ways:

- Downtime that drags on, because nobody can find the cause quickly.
- Breaches that go unnoticed, since a hijacked DNS or rogue device can quietly redirect or watch traffic for weeks.
- No way to tell an outage from an attack, so people restart routers and wait, while the real problem continues.

A simple, understandable diagnostic tool lets everyday users spot and fix problems early instead of relying on expert help they do not have.

> **Note:** Teams may use sample or self-generated packet captures from a lab or simulated setup. They do not need access to any real production network. Include a few sample captures in your repository so the judges can run your tool without setting up a network.

# Submission Guidelines

## 1. What to Submit

Submit a **public** GitHub repository along with a demo video of 2 to 5 minutes. The demo video must be either added to the repository or linked in the README. If you link it, make sure it is publicly accessible (GitHub blocks files over 100 MB).

## 2. README

Include a README that explains the problem, your solution, the features and the setup instructions. The tool should run with a clearly documented setup, so judges can try it without any guesswork.

### 3. Originality

The project must be built during the hackathon. Teams must list every open source library or API they use in the README. Libraries for reading captures (such as Scapy or PyShark) are allowed, but the detection and diagnosis logic must be your own. Simply wrapping the output of an existing tool such as Wireshark or tshark will not count.

### 4. Safe Testing

All testing must stay within a lab, simulated or sample environment. Test your tool only on sample captures or on networks you own or have permission to use. Teams must not capture or analyze traffic on networks they do not own or have permission to use.

### 5. Clean Code

Validate all input, protect user data and do not hardcode passwords, keys or other secrets in the repository.

### 6. Evaluation

All valid submissions will be scored by the judging panel using the criteria given below.

## Terms and Conditions

### 1. Ownership

Teams keep ownership of their projects. Organizers may ask for permission before showing a project in promotional material.

### 2. Disqualification

Any team that breaks the rules or engages in misconduct will be disqualified.

### 3. Liability

Organizers are not responsible for technical failures, data loss or equipment problems during the event.

### 4. Responsible Use

Everything built and shown during this event must be used responsibly. Tools must not be used to attack, disrupt or spy on any network or person.

# Judging Criteria

| Criteria | Weight | What the judges look at |
|---|---|---|
| **Novelty** | **20%** | Originality of the idea and how different it is from existing tools. |
| **Technical Implementation** | **30%** | Depth of the solution, how much of the detection logic is the team's own, code quality, structure and handling of edge cases. |
| **Security of the Solution** | **20%** | Whether it handles input safely, avoids leaking data and avoids hardcoded secrets. |
| **Feasibility** | **10%** | How practical, reliable and usable the solution is in the real world, including how easy it is to set up and run from the README. |
| **Video Presentation** | **20%** | A clear demo that explains the problem, the solution and the key features. |

**Additional Notes:** Projects that are unfinished or do not work may lose marks, and any demo that is faked or misleading will result in disqualification.